

Retrievers in LangChain

1. What is a Retriever?

- **Definition:**

A retriever is a **component in LangChain that fetches relevant documents from a data source in response to a user's query.**

- **How it works (flow):**

1. **User query** → Retriever.
2. Retriever searches in a **data source** (vector store, API, Wikipedia, etc.).
3. Finds **most relevant documents**.
4. Returns them as **LangChain Document objects**.

👉 Think of a retriever as a **search engine** inside your RAG pipeline.

- **Important:**

- Not just one retriever → many types exist.
- All retrievers are **runnables** (like models, prompts).
 - Means you can call `.invoke()` on them.
 - Easy to plug into chains or pipelines.

2. Why Are Retrievers Important?

- **Core component** of RAG (4th after: Document Loader, Text Splitter, Vector Store).
- Needed to **fetch precise context** for LLMs.
- Enable **better accuracy** and **smarter search strategies**.

👉 Without good retrievers, RAG pipelines give poor or redundant results.

3. Categories of Retrievers

Retrievers can be classified in **two ways**:

- ◆ **A. Based on Data Source**

- Work with **different data sources**. Examples:
 1. **WikipediaRetriever** – Queries Wikipedia API for articles.
 2. **VectorStoreRetriever** – Fetches from a vector store (FAISS, Chroma, Weaviate).
 3. **ArxivRetriever** – Fetches research papers from Arxiv.

👉 Each retriever is specialized for its source.

- ◆ **B. Based on Search Strategy**

- Different retrievers use **different algorithms** to select documents. Examples:
 1. **Similarity Search Retriever** – Basic semantic similarity.
 2. **MMR (Maximum Marginal Relevance) Retriever** – Fetches **relevant + diverse** results (avoids redundancy).
 3. **Multi-Query Retriever** – Expands ambiguous query into multiple queries via LLM → merges results.
 4. **Contextual Compression Retriever** – Compresses docs after retrieval → keeps only query-relevant parts.
 5. **Other retrievers:** Parent Document Retriever, Time-weighted Vector Retriever, Self-query Retriever, Ensemble Retriever, etc.

4. Key Retrievers in Detail

- ◆ **4.1 Wikipedia Retriever**

- Fetches relevant articles from Wikipedia API.
- Works via **keyword matching**, not semantic embeddings.

- Returns results as Document objects.

Code Example:

```
from langchain_community.retrievers import WikipediaRetriever

retriever = WikipediaRetriever(top_k_results=2, lang="en")
results = retriever.invoke("Albert Einstein")
for doc in results:
    print(doc.page_content[:200])
```

◆ 4.2 Vector Store Retriever

- Most **common retriever** in LangChain.
- Works on **semantic similarity search** using embeddings.

How it works:

1. Documents → embeddings → stored in vector DB.
2. Query → converted to vector.
3. Perform similarity search.
4. Fetch top-K relevant docs.

Code Example:

```
from langchain_openai import OpenAIEmbeddings
from langchain.vectorstores import Chroma

embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_texts(
    ["LangChain is a framework", "Chroma is a vector store"],
    embedding=embeddings
)

retriever = vectorstore.as_retriever(search_kwargs={"k":2})
docs = retriever.invoke("What is Chroma used for?")
```

👉 Note: Could also use `vectorstore.similarity_search()`, but retriever objects are better since they can integrate with **advanced search strategies**.

◆ 4.3 Maximum Marginal Relevance (MMR) Retriever

- **Problem:** Normal retrievers return duplicate results.
- **Solution:** MMR ensures results are **relevant + diverse**.

Logic:

- Pick first most relevant doc.
- Next picks doc that is **both relevant & dissimilar** from already selected ones.
- Reduces redundancy.

Code Example:

```
retriever = vectorstore.as_retriever(
    search_type="mmr",
    search_kwargs={"k":3, "lambda_mult":0.5}
)
```

- `lambda_mult`:
 - 1.0 → behaves like normal similarity search.
 - 0.0 → maximum diversity.

◆ 4.4 Multi-Query Retriever

- **Problem:** User queries can be **ambiguous**.
 - Example: “How can I stay healthy?” → diet? exercise? stress management?
- **Solution:** Expands query into multiple variations via an **LLM**, retrieves results for each, merges them.

Code Example:

```
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain_openai import ChatOpenAI

llm = ChatOpenAI()
retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(),
    llm=llm
)

docs = retriever.invoke("How can I stay healthy?")
```

👉 Improves retrieval when queries are **broad or ambiguous**.

◆ 4.5 Contextual Compression Retriever

- **Problem:** Retrieved docs often contain **irrelevant info**.
- **Solution:** Compress docs → keep only query-relevant sentences.
- Uses **LLM as a compressor**.

Code Example:

```

from langchain.retrievers import ContextualCompressionRetriever
from langchain.chains import LLMChainExtractor
from langchain_openai import ChatOpenAI

base_retriever = vectorstore.as_retriever()
compressor = LLMChainExtractor.from_llm(ChatOpenAI())

retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=base_retriever
)

docs = retriever.invoke("What is photosynthesis?")

```

👉 Best for long or mixed-topic documents.

5. Other Retrievers (not covered in detail)

- **Parent Document Retriever** – Retrieves parent docs instead of chunks.
- **Time-Weighted Vector Retriever** – Prioritizes recent docs.
- **Self-Query Retriever** – Parses query into structured filters.
- **Ensemble Retriever** – Combines multiple retrievers.

6. Best Practices

- Use **vanilla vector store retrievers** for basic RAG.
- Use **MMR retriever** when redundancy is a problem.
- Use **Multi-query retriever** when queries are ambiguous.
- Use **Contextual compression retriever** for long or noisy docs.
- Experiment with **advanced retrievers** for better accuracy in production.

7. Interview Q&A

Q1. What is a retriever in LangChain?

👉 A component that fetches relevant documents from a data source in response to a user query.

Q2. How is a retriever different from a vector store?

👉 Vector store stores embeddings; retriever is a search engine on top of it.

Q3. Why use retrievers instead of direct similarity search?

👉 Retrievers allow **advanced strategies** (MMR, multi-query, compression) + they're **runnables** (easy to integrate in chains).

Q4. Explain MMR retriever.

👉 Fetches results that are both **relevant and diverse**, avoids redundant results.

Q5. Explain Multi-Query retriever.

👉 Expands ambiguous queries into multiple variations using an LLM → merges results.

Q6. When would you use Contextual Compression retriever?

👉 When documents are long/mixed → need only relevant sentences.

8. Real-World Applications

- **Chat with PDFs/reports/books** – Accurate retrieval for RAG pipelines.
- **Legal search engines** – Diverse case references (MMR).
- **Healthcare assistants** – Disambiguating broad patient queries (Multi-query).
- **Customer support bots** – Trim irrelevant info in answers (Compression).

- **News/research summarization** – Fetch diverse sources.
-

✓ **Summary:**

- Retrievers = Search engines in RAG.
- Types: Based on **data source** or **search strategy**.
- Key ones: Wikipedia, VectorStore, MMR, Multi-query, Compression.
- Core for improving RAG **accuracy and reliability**.